

CGA: Accelerating BFS Through an Sparsity-Aware Adaptive Framework on Heterogeneous Platforms

Lei Xu^{ID}, Haipeng Jia^{ID}, and Yunquan Zhang^{ID}, *Senior Member, IEEE*

Abstract—Direction optimization determines whether to use Sparse Matrix-Sparse Vector Multiplication (SpMSpV) or Sparse Matrix-Dense Vector Multiplication (SpMV) based on the input vector's sparsity at each iteration of Breadth-First Search (BFS), aiming to achieve the fastest graph traversal. Although prior work on direction optimization has achieved state-of-the-art performance on either CPUs or GPUs, it has not fully leveraged the capabilities of modern heterogeneous platforms. This is because SpMSpV/SpMV execution times on GPUs do not consistently outperform those on CPUs, particularly for SpMSpV. In response, this paper introduces **CGA**, a machine learning-based adaptive framework for BFS that optimally selects between CPU and GPU kernels, effectively **Adapting** to diverse real-world graphs, vectors, and computing platforms. Our contributions include a novel set of bucket-based SpMSpV algorithms that significantly enhance kernel performance in high-sparsity scenarios, along with a low-overhead decision tree model and reduced CPU-GPU data transfers. Experimental results show that our framework outperforms previous state-of-the-art methods, achieving up to a 4.91x speedup over CPU-only baseline and 3.27x speedup over GPU-only baseline.

Index Terms—Sparse matrix-sparse vector multiplication (SpMSpV), sparse matrix-dense vector multiplication (SpMV), multi-core CPU, GPU, adaptive performance optimization, machine learning.

I. INTRODUCTION

BREADTH-FIRST Search (BFS) is a cornerstone algorithm in graph processing, widely used across diverse domains such as network analysis, text analytics, computational biology, and social network modeling [1], [2], [3], [4], [5]. It serves as the foundation for numerous graph algorithms, including single-source shortest paths [6], PageRank [7], and betweenness centrality [8]. Due to its widespread adoption, BFS is a critical benchmark in graph processing suites such as Graph500 [9] and GAP [10], driving ongoing efforts to enhance its performance.

Received 18 November 2024; revised 16 August 2025; accepted 16 October 2025. Date of publication 22 October 2025; date of current version 20 November 2025. This work was supported in part by the National Key Research & Development Program of China under Grant 2023YFB3001700, in part by the National Natural Science Foundation of China under Grant 62372432, Grant 61972376, Grant 62072431, and Grant 62032023, and in part by Shanxi Province Science and Technology Major Special Project under Grant 202201010101004. Recommended for acceptance by J. Nicod. (*Corresponding author: Haipeng Jia.*)

Lei Xu is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, and also with the University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: cq_xulei@foxmail.com).

Haipeng Jia and Yunquan Zhang are with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China (e-mail: jia-haipeng@ict.ac.cn; zyq@ict.ac.cn).

Digital Object Identifier 10.1109/TPDS.2025.3624289

BFS operates iteratively, discovering the next set of active vertices (next frontier) from the current set (current frontier) by traversing their connecting edges. This process continues until all vertices have been explored. Mathematically, each BFS iteration can be modeled as a sparse matrix-vector multiplication (SpMV), where the adjacency matrix is stored in a sparse format and the current frontier is represented as a dense vector [11]. Optimizing SpMV has led to significant performance gains for BFS across various computing platforms [4], [12], [13], [14], [15]. However, in real-world applications, the frontier often contains numerous inactive vertices (zeros), leading to redundant computations when using SpMV [12], [16].

To address this inefficiency, Sparse Matrix-Sparse Vector multiplication (SpMSpV) is employed. By storing the frontier in a sparse format, SpMSpV eliminates unnecessary computations and outperforms SpMV in scenarios with high sparsity. This capability has driven the adoption of direction optimization, a technique that dynamically selects between SpMV and SpMSpV based on the sparsity of the input vector [17].

Initial research on direction optimization primarily targeted shared-memory multicore CPUs [15], [17], [18]. With the rise of heterogeneous computing, particularly GPUs, significant progress has been made in optimizing sparse matrix operations for these platforms. High-performance implementations of SpMSpV [16], [19] and SpMV [20], [21] have been developed, extending direction optimization to GPUs [12], [22], [23]. Despite these advancements, the full potential of heterogeneous computing remains untapped, as GPU-based sparse operations do not consistently outperform their CPU counterparts—particularly for SpMSpV. Unlocking this potential requires not only leveraging sparsity but also accounting for the computational affinity of SpMV and SpMSpV across CPUs and GPUs.

In this paper, we address these challenges by proposing **CGA**, a machine learning-based adaptive framework for BFS that optimally integrates the strengths of CPUs and GPUs on heterogeneous platforms. Our contributions are as follows:

- We propose a set of bucket-based masked SpMSpV algorithms for multicore CPUs, which innovatively incorporate diverse mask fusion strategies and accumulator variants. These algorithms improve performance under high sparsity and complement existing work in this area.
- We develop a learning-based framework with four decision models that adaptively select CPU or GPU kernels. Unlike prior work, it effectively exploits heterogeneous platforms through coordinated model decisions.

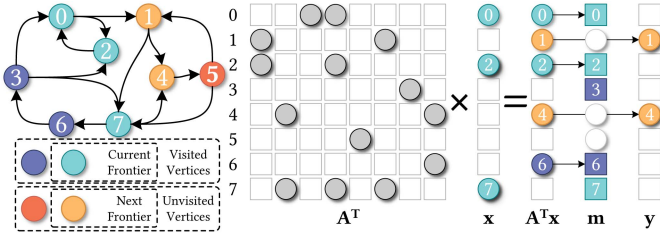


Fig. 1. Matrix-Vector Multiply and Graph Traversal.

Algorithm 1: BFS Using Masked Matrix-Vector Multiplication.

Require: Matrix A^T , starting vertex s **Ensure:** All vertices reachable from s are visited

- 1: Initialize the ‘Current Frontier’ vector x
 - 2: Initialize the vector `visited` with all entries 0
 - 3: Set `visited`[s] = 1
 - 4: Initialize a scalar $depth = 0$
 - 5: **repeat**
 - 6: $m = \neg \text{visited}$ // the vector `unvisited`
 - 7: $y \langle m \rangle = A^T \times x$ //Masked matrix-vector multiplication
 - 8: **for** each non-zero i in y **do**
 - 9: Set `visited`[i] = $depth + 1$
 - 10: **end for**
 - 11: Increment $depth$; $x = y$
 - 12: **until** $|x| == 0$ //No nodes are in ‘Current Frontier’
-

- We demonstrate the superior performance of our framework on modern heterogeneous platforms, achieving up to a 4.91x speedup over CPU-only state-of-the-art method and a 3.27x speedup over GPU-only method.

This work extends our previous conference version [24] by introducing two major innovations: (1) replacing an expert-defined, criteria-based decision process with a machine learning approach for more accurate kernel selection, and (2) presenting CGA, a novel framework that fully exploits heterogeneous computing platforms by synergizing CPU and GPU resources. Additionally, the bucket-based SpMSpV algorithms from the conference version serve as key building blocks within CGA, contributing significantly to BFS’s overall performance improvements.

II. BACKGROUND AND MOTIVATION

A. Matrix-Vector Multiply and Graph Traversal

Adjacency matrices are a fundamental representation of graph data, providing a direct bridge between graph structures and matrix operations [11]. This connection is particularly pronounced in matrix-vector multiplication, which underpins each iteration of BFS traversal. As shown in Fig. 1, multiplying the current frontier vector x by the transposed adjacency matrix A^T identifies the nodes adjacent to the current frontier ($A^T x$). To prevent revisiting vertices, a mask of unvisited vertices filters $A^T x$, yielding the next frontier vector y , expressed as $y \langle m \rangle = A^T x$.

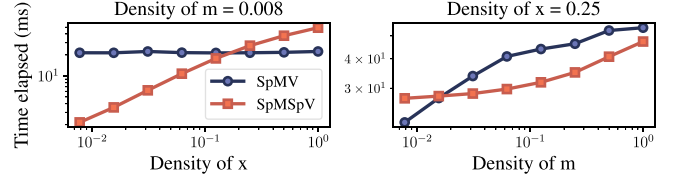


Fig. 2. The impact of input and mask vector densities on the SpMSpV/SpMV.

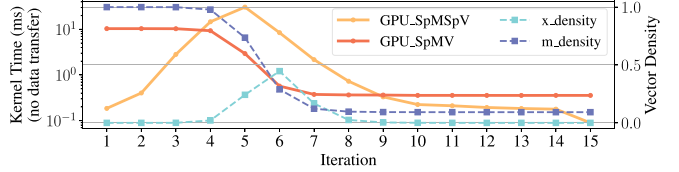


Fig. 3. BFS Iteration Process on soc-LiveJournal1.

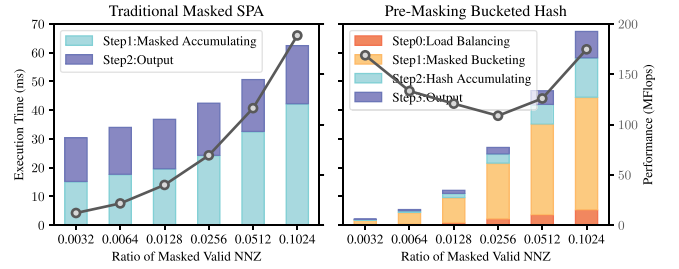


Fig. 4. Performance comparison between the traditional masked SPA-based SpMSpV and the proposed Pre-Masking Bucketed Hash under low sparsity conditions. The proposed algorithm demonstrates significantly higher performance when the ratio of masked valid non-zero elements is low.

Algorithm 1 outlines the BFS algorithm’s implementation through masked matrix-vector multiplication. The iterative traversal of the graph is achieved by repeatedly executing the core multiplication operation specified in line 7 of the algorithm. In this paper, the term “kernel” refers to the computational unit responsible for performing this multiplication operation, which is fundamental to BFS execution.

B. Motivation: SpMSpV Vs. SpMV

The ideal computational complexity of the kernel is expected to be: $\mathcal{O}(\text{masked_valid_nnz})$, where *masked valid nnz* represents the number of non-zero elements involved in the final result accumulation, ensuring the elimination of redundant computations. Under uniform distribution, this is approximately equal to $nnz \times x_density \times m_density$, where *nnz* denotes the total number of non-zero elements in the matrix A , while *x_density* and *m_density* represent the densities (ratios of non-zero elements) in the vectors x (current frontier) and m (mask), respectively.

In practice, the ideal complexity is unattainable due to execution inefficiencies and algorithmic limitations. As shown in Fig. 2, SpMSpV and SpMV differ in how they exploit input sparsity and handle masks. SpMSpV skips zero entries in x , making it efficient when *x_density* is low, but its execution time is still bounded by $x_density \times nnz$, limiting gains under

high sparsity. In contrast, SpMV assumes a dense x and ignores sparsity, but achieves better throughput when $x_density$ is high and benefits from row-wise filtering by the mask m , yielding a near-linear relation to $m_density$.

During BFS, both $x_density$ and $m_density$ vary across iterations, making operator selection between SpMSpV and SpMV critical to performance. Fig. 3 illustrates this on a real-world graph, where a 15-iteration traversal shows significant fluctuations in x and m densities, resulting in varying performance of the two operators. Addressing this dynamic behavior is a key focus of this work, building on prior efforts in operator selection for BFS [17], [22], [25].

C. Challenges and Design Issues

While prior work has explored operator selection in BFS, our analysis reveals that existing SpMSpV and SpMV strategies still leave significant performance untapped, particularly on modern heterogeneous platforms. To fully exploit hardware capabilities and maximize BFS throughput, three key challenges must be addressed:

1) *High-Performance Operators for Ultra-High Sparsity:* During BFS traversal, the number of valid non-zero elements is often extremely low, posing challenges for traditional SpMSpV algorithms. As shown in the left panel of Fig. 4, the performance of the traditional method deteriorates sharply as the ratio of masked valid non-zero elements decreases.

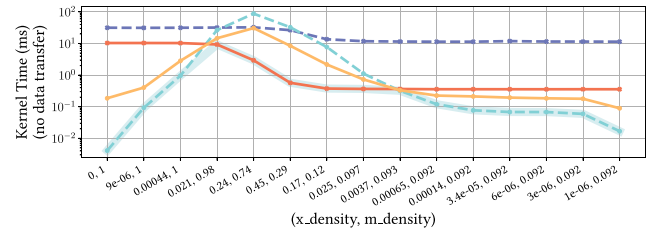
The Sparse Accumulator (SPA), a dense array used in SpMSpV, suffers from poor cache efficiency due to the accumulation of zero elements under ultra-high sparsity. While compact storage methods like hash tables can reduce this issue, they often introduce false sharing, degrading parallel efficiency on multicore processors. Additionally, during the output phase, many zero elements force these methods to perform $\mathcal{O}(n)$ operations instead of the ideal $\mathcal{O}(\text{masked_valid_nnz})$.

To tackle these issues, we introduce a masked bucketing step, shown in the right panel of Fig. 4, which delivers high performance even under conditions of extreme sparsity.

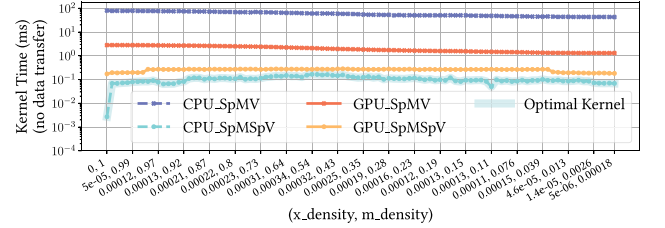
2) *Heterogeneous Kernel Decision:* Efficient BFS execution requires selecting the most suitable kernel for each iteration. However, determining the optimal operator on heterogeneous platforms is inherently complex.

One key challenge arises from variations in vector density across different graph types. As illustrated in Fig. 5, kernel execution times vary significantly for two common graph categories: scale-free and mesh graphs. Scale-free graphs, such as social networks, follow a power-law degree distribution, leading to fluctuating $x_density$ that rises and falls throughout traversal. In contrast, mesh networks, derived from spatial data like road maps, exhibit consistently low $x_density$ due to their high diameter and low-degree distributions, requiring more iterations to complete traversal.

Platform-specific performance further complicates selection. GPUs, while effective for high-density computations, often fail to achieve optimal performance at low $x_density$, where their throughput advantages diminish [26]. Additionally, SpMSpV involves a k-way merge operation, which tends to be less efficient

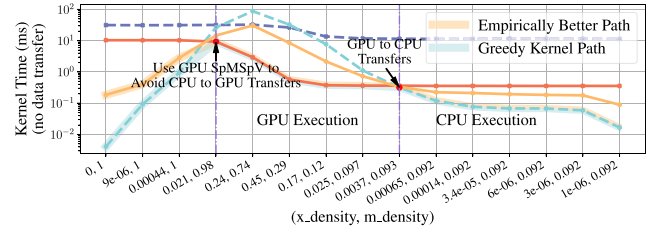


(a) Performance on **soc-LiveJournal1** (scale-free graph)

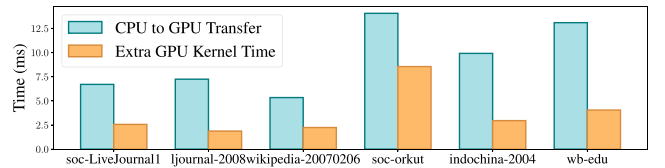


(b) Performance on **road_usa** (mesh graph)

Fig. 5. Performance Comparison of SpMSpV/SpMV for Graph Traversal: Operations Across different BFS iterations.



(a) Illustration of the greedy kernel path versus the empirically better path.



(b) GPU computation overhead vs. data transfer cost on scale-free graphs. Extra GPU kernel time is the additional time spent running SpMSpV on GPU instead of CPU during early iterations.

Fig. 6. Comparison between the greedy kernel and empirically better path.

when implemented on GPUs [19]. Consequently, effectively leveraging the complementary strengths of both CPU and GPU resources is critical to maximizing BFS throughput.

3) *Runtime Overhead:* Heterogeneous adaptive scheduling introduces additional runtime overhead, primarily due to data transfers between the CPU and GPU. As shown in Fig. 6(a), the Greedy Kernel Path selects the kernel with the shortest execution time at each step. However, in the case of scale-free graphs, this strategy typically leads to one CPU-to-GPU transfer and one GPU-to-CPU transfer. The overhead of these transfers can outweigh the performance gains from locally optimal choices. The Empirically Better Path adopts a different strategy by using GPU SpMSpV in the early stages to avoid one data transfer.

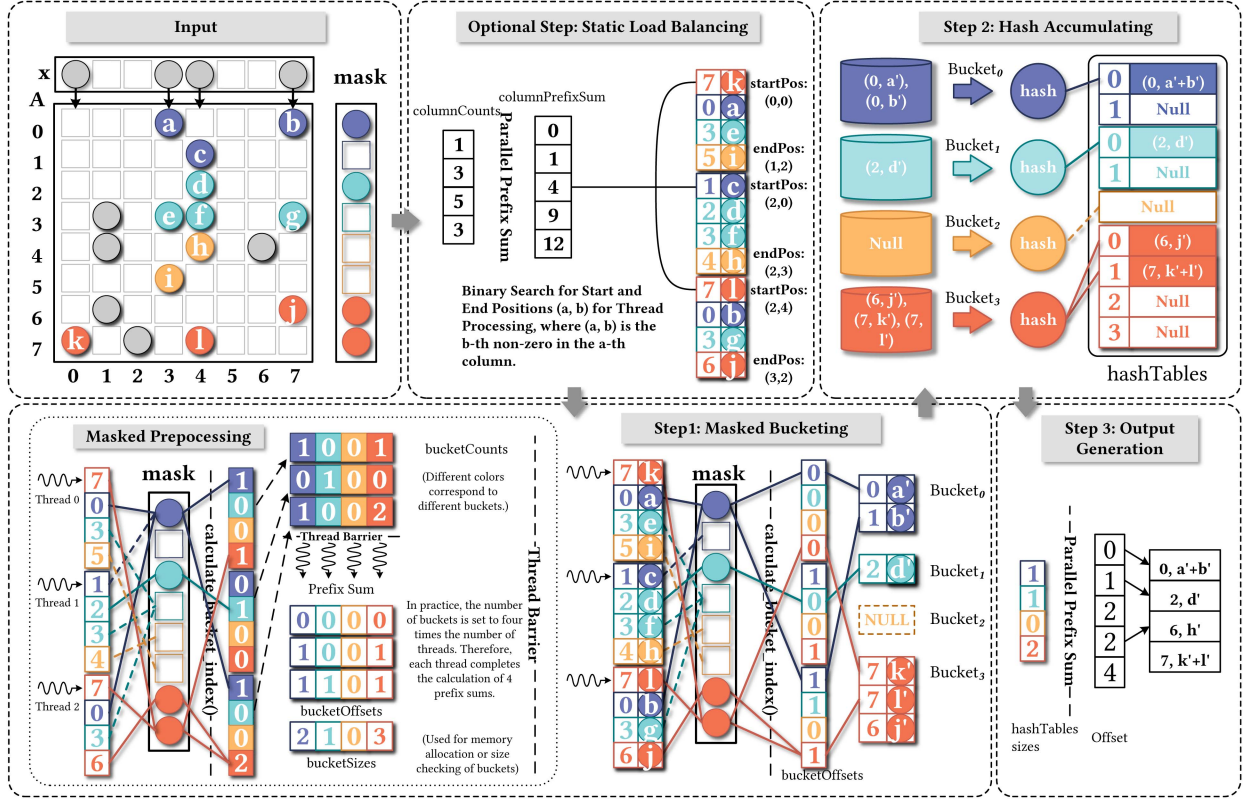


Fig. 7. Workflow Diagram of the Pre-Masking Bucketed Hash-based SpMSpV.

This adjustment leads to faster overall traversal. The final GPU-to-CPU transfer remains both unavoidable and acceptable for two reasons: first, as the number of visited vertices increases, the computational workload naturally diminishes near the end of traversal, making CPU execution more efficient during these final iterations; second, the final results usually be stored in CPU memory.

Fig.6(b) compares the extra time incurred by using a GPU SpMSpV kernel (instead of the locally optimal CPU kernel) in early BFS iterations, against the cost of CPU-to-GPU data transfers. The results show that the computation overhead remains consistently lower than the transfer cost, leading to overall performance gains across different scale-free graphs.

Beyond data transfers, runtime overhead also includes decision-making for kernel selection and format conversions between sparse and dense vector representations. These factors underscore the importance of optimizing these processes to minimize overhead and improve overall performance.

III. PARALLEL MASKED SPMSPVs

To address the inefficiency of traditional SPA-based approaches under high sparsity, we propose a set of bucket-based masked SpMSpV algorithms that incorporate novel combinations of mask fusion strategies and accumulator designs. These innovations enable efficient multithreaded execution by avoiding dense array overhead and mitigating contention in concurrent

accumulation, and are further extended with different algorithmic variants tailored for load imbalance and varying sparsity levels.

A. Algorithm Overview

This section introduces a masked SpMSpV algorithm optimized for scenarios with very low *masked_valid_nnz*, as shown in Fig.7. As discussed in Section II-C1, the SPA-based method becomes inefficient during the accumulation phase due to its dense array structure. To address this, a hash table is utilized for accumulation. However, multithreaded writes to hash tables can lead to data races and false sharing, significantly degrading performance. Our proposed algorithm employs a bucket strategy to circumvent these issues, integrating the masking step directly into the initial bucketing process to reduce computational overhead. The algorithm comprises the following four steps:

Optional Step: Static Load Balancing To mitigate potential load imbalances caused by uneven distributions of non-zero elements, the algorithm offers an optional static load balancing step. This step incurs overhead, requiring a trade-off between its costs and benefits. For simplicity, this step is omitted in Algorithm 2, where tasks are divided directly by column.

Fig.7 shows this static balancing approach. Tasks are distributed based on the prefix sum (*colPrefixSum*) of column element counts (*colCounts*). Each thread determines its task

boundaries using binary search on *colPrefixSum*, ensuring an approximately even distribution of non-zero elements across threads.

Masked Bucketing (lines 3–11 of Algorithm 2) In this step, the algorithm first filters $\mathbf{A}(i, j)$ based on the mask, then inserts the scaled non-zero elements $\mathbf{A}'(i, j) = \mathbf{A}(i, j) \times \mathbf{x}(j)$ (Line 7) into different buckets (Line 8). Elements with neighboring row indices i are grouped into the same bucket. The bucket index is calculated using the formula: $\lfloor (n_bucket \times (i + 1) - 1) / m \rfloor$, where n_bucket is the number of buckets, and m is the number of rows of the matrix as well as the length of the mask vector. To improve the effectiveness of dynamic load balancing, the number of buckets is typically set to four times the number of threads.

To avoid data race from multiple threads inserting into the same bucket, the algorithm employs the array *bucketOffsets*[], which records the starting insertion position for each bucket. As illustrated in Fig. 7, this array is precomputed during a Masked Preprocessing step, which determines the number of elements in each bucket (*bucketCounts*[]) and computes prefix sums to derive *bucketOffsets*[],. The final element of *bucketOffsets*[] indicates the total number of elements across all buckets, aiding in memory allocation.

This bucketing approach is lock-free, with only two instances of thread synchronization, resulting in high parallel efficiency.

Hash Accumulating (lines 13–20 of Algorithm 2)

Following the masked bucketing step, the non-zero elements exhibit the following properties:

- 1) All elements within a bucket are included in the result, eliminating the need for mask checks during accumulation.
- 2) Row indices within each bucket are close together, improving spatial locality during accumulation.
- 3) Different buckets contain distinct row indices, avoiding interference between threads.

Therefore, the accumulation phase is straightforward: each thread processes its assigned bucket using a private hash table. Non-zero row indices serve as keys, while their scaled values are stored as values. The hash function is defined as:

$$hash = \text{mod}(257 \times i, hash_capacity),$$

with linear probing [27] used for conflict resolution. Each thread dynamically allocates its hash table based on the bucket size (*hash_capacity*), which is a power of two slightly larger than the number of intermediate products in the bucket (computed using the function *calculate_hash_size*, Line 14). This enables the hash function to be efficiently implemented using bitwise operations. Dynamic task scheduling is employed to balance workloads across buckets of varying sizes.

Output Generation (lines 22–28 of Algorithm 2) In the final step, the sparse result vector is generated. Each hash table maintains a count of its non-zero elements. By performing a prefix sum on these counts (Line 22), the total number of elements in the result vector is determined, and the required memory is allocated. Using offsets obtained from the prefix sum, multiple threads insert elements into the result vector bucket by bucket (Line 26).

Algorithm 2: Pre-Masking Bucketing Hash-based SpM-SpV.

Require: Sparse matrix $\mathbf{A}$, vector $\mathbf{x}$, mask vector *mask*
Ensure: Result vector *y*

```

1: /* Step 1: Masked Bucketing (Preprocessing omitted) */
2: /* Tasks are distributed by column without load
   balancing. */
3: for each non-zero  $\mathbf{x}(j)$  in  $\mathbf{x}(\cdot)$  in parallel do
4:   for each non-zero  $\mathbf{A}(i, j)$  in  $\mathbf{A}(\cdot, j)$  do
5:     if mask[i]  $\neq 0$  then
6:        $k \leftarrow \text{calculate\_bucket\_index}(i)$ 
7:        $\mathbf{A}'(i, j) \leftarrow \mathbf{A}(i, j) \times \mathbf{x}(j)$ 
8:       insert (Bucketk, (i,  $\mathbf{A}'(i, j)$ ))
9:     end if
10:   end for
11: end for
12: Step 2: /* Hash Accumulating */
13: for Bucketk in Buckets in parallel do
14:   hash_capacity  $\leftarrow$ 
     calculate_hash_size(size(Bucketk))
15:   hashTablek  $\leftarrow$  create_hash_table(hash_capacity)
16:   for every pair (i,  $\mathbf{A}'(i, j)$ ) in Bucketk do
17:     hashTablek.INSERT(i,  $\mathbf{A}'(i, j)$ )
18:   end for
19:   Offset[k]  $\leftarrow$  size(hashTablek)
20: end for
21: /* Step 3: Output Generation */
22: parallel_prefix_sum (Offset[:])
23: for hashTablek in hashTables in parallel do
24:   i  $\leftarrow 0$ 
25:   for every pair (idx, value) in hashTablek do
26:     y[Offset[k] + i]  $\leftarrow$  (idx, value); i  $\leftarrow i + 1$ 
27:   end for
28: end for
29: return y

```

B. Algorithm Variants

The proposed algorithm, referred to as the Pre-Masking Bucketed Hash-based SpM-SpV (PM-BHash), is designed to optimize performance under high sparsity. However, the static load balancing step introduces additional overhead, necessitating a trade-off between its costs and benefits. To address this, we define two variants:

- *PM-BHash*: The non-load-balanced version.
- *LB-PM-BHash*: The load-balanced version, which incorporates static load balancing.

Building on this foundation, we developed a variant called the Pre-Bucketing Masked SPA-based SpM-SpV (PB-MSPA), which is designed for scenarios with high *m_density*, where the result vector contains many nonzeros. In such cases, the SPA-based method offers better accumulation efficiency than hash-based approaches.

The core workflow of PB-MSPA is illustrated in Fig. 8. The algorithm begins with a bucketing phase, after which the SPAState array is initialized based on the mask: state 0

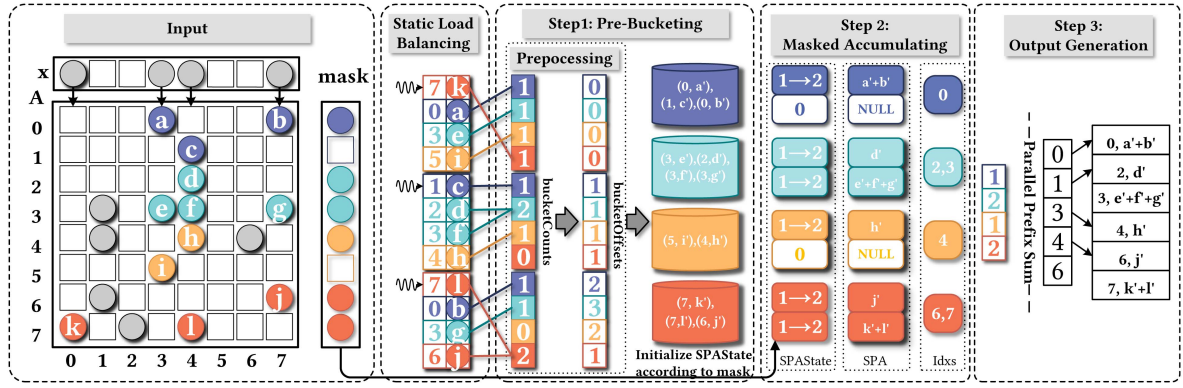


Fig. 8. Workflow Diagram of the Pre-Bucketing Masked SPA-based SpMSpV.

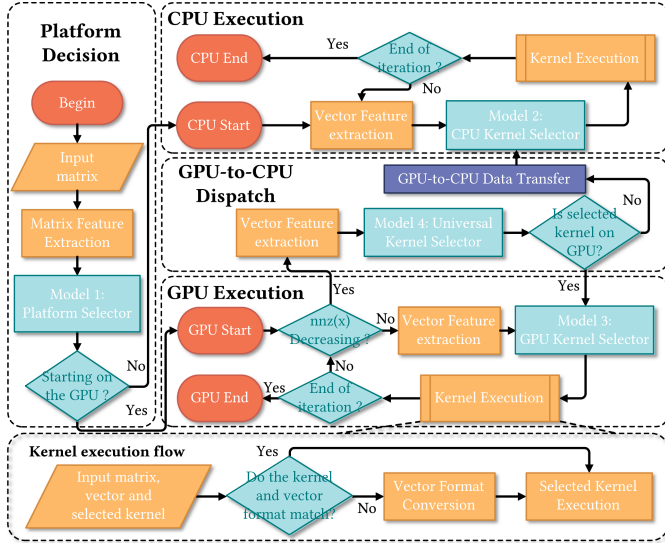


Fig. 9. Flowchart of Adaptive Graph Traversal with a Single Data Transfer.

indicates no computation, while state 1 marks entries that require computation but have not yet accumulated a value. During the accumulation phase, state 1 transitions to state 2, signaling that a partial sum is stored in the SPA. As accumulation proceeds, the corresponding indices are recorded in Idxs. Finally, the result vector is constructed by scanning Idxs and retrieving values from the SPA.

The mask of this algorithm is not applied during bucketing but is instead deferred to the accumulation phase. This design avoids redundant mask accesses during bucketing. Although this may introduce minor redundancy in the buckets, the effect is negligible when $m_density$ is high.

Like the hash-based variant, PB-MSPA includes two configurations:

- *PB-MSPA*: The non-load-balanced version.
- *LB-PB-MSPA*: The load-balanced version with static load balancing.

These variants provide flexibility for adapting to different levels of sparsity and masking densities, enabling optimized performance across diverse graph processing scenarios.

IV. CGA: ADAPTIVE FRAMEWORK

This section provides a detailed overview of the CGA adaptive framework, which employs a learning-based approach with four coordinated decision models for adaptive kernel selection. Unlike prior approaches that focus on selecting among algorithms within a single device (either CPU or GPU), CGA can dynamically leverage both processing units, fully exploiting their complementary strengths. It comprises two key stages: the **off-line model training** and the **on-line inference execution**. The off-line model training is performed once on a specific platform, generating four essential models. These models can be trained using diverse datasets and hardware parameters, enabling them to adapt to varying conditions and make accurate decisions. Once trained, the framework provides precise guidance for platform and algorithm selection with minimal runtime overhead across various graph datasets during the on-line inference execution.

A. One-Copy Execution Flow

Before introducing model training and inference execution, we present the One-Copy Execution Flow, which defines the functional requirements of the four models during training and outlines the decision process for inference execution. This flow is based on the empirical observation that in many graph traversal algorithms, the sparsity ratio ($x_density$) follows a global trend with at most one significant decreasing phase—particularly on scale-free graphs. To efficiently exploit this property while minimizing data transfer overhead, the execution flow allows at most one GPU-to-CPU data transfer.

This single-transfer design balances adaptivity and efficiency: for mesh graphs with stable $x_density$, the entire traversal is executed on either the CPU or GPU branch; for scale-free graphs, the framework dynamically switches from GPU to CPU at the optimal point, if needed. For graphs with unknown structures, the One-Copy strategy serves as a conservative yet effective policy, as multiple transfers often introduce overhead exceeding that of executing all iterations on the GPU alone.

The One-Copy Execution Flow, illustrated in Fig. 9, consists of four steps: Platform Decision, CPU Execution, GPU Execution, and GPU-to-CPU Dispatch.

1) *Platform Decision*: The execution flow begins by selecting the backend platform—CPU or GPU—for the first kernel

execution. This decision is guided by **Model 1: Platform Selector**. The primary role of Model 1 is to differentiate between mesh graphs, which are always executed on the CPU, and scale-free graphs, where execution typically starts on the GPU to achieve the optimal path.

Model 1 relies solely on matrix features, making its task relatively straightforward and resulting in high prediction accuracy. By focusing on identifying graph types, this model ensures a robust starting point for the execution flow.

2) *CPU Execution*: If Model 1 selects the CPU as the backend, the entire graph traversal process proceeds on the CPU. Kernel selection during CPU execution is managed by **Model 2: CPU Kernel Selector**, which determines the CPU optimal kernel for each BFS iteration.

The input to Model 2 includes not only matrix information but also features derived from both the input vector and the mask vector. To enhance accuracy, the model incorporates key features that differentiate the candidate CPU kernels. These features are further detailed in the model training section.

The CPU Execution also addresses heterogeneous platform scenarios. On systems with only multi-core processors, Model 2 alone suffices for adaptive kernel selection.

3) *GPU Execution*: If GPU execution is chosen, the traversal process follows an iterative prediction-execution cycle managed by **Model 3: GPU Kernel Selector**. Unlike CPU execution, GPU execution introduces an additional criterion: detecting whether $nnz(\mathbf{x})$ (the number of non-zero elements in the input vector $\mathbf{x}$) is decreasing. This criterion determines when to initiate a GPU-to-CPU transfer, ensuring that the first few iterations remain on the GPU for optimal performance, even as $nnz(\mathbf{x})$ diminishes.

To evaluate this criterion, input vectors from recent iterations are compared. While Model 3 focuses solely on predicting the optimal GPU kernel for the current iteration using current vector information, the $nnz(\mathbf{x})$ criterion operates independently of Model 3's scope.

During execution, switching between SpMV and SpMSpV kernels may introduce a mismatch between the predicted kernel and the current input vector format. Consequently, format conversion of the vector may be required, as depicted in the “Kernel Execution Flow” of Fig.9.

4) *GPU-to-CPU Dispatch*: This stage determines whether to proceed with the single GPU-to-CPU data transfer. The decision is made by **Model 4: Universal Kernel Selector**, which evaluates all available CPU and GPU kernel options. If Model 4 selects a GPU kernel, the process remains on the GPU. Otherwise, a One-Copy data transfer is triggered, where the input vectors $\mathbf{m}$ and $\mathbf{x}$ are copied from GPU to host memory—providing all necessary inputs for CPU execution, as the matrix is already mirrored in both memories. Execution then transitions to the CPU branch, proceeding under the control of Model 2.

B. Candidate Algorithms

This section explores the candidate algorithms considered for selection by the models in CGA. Alongside the bucket-based SpMSpV algorithms discussed in Section III, several

TABLE I
OVERVIEW OF THE CANDIDATE ALGORITHMS

Category	Algorithm	Abbr.
CPU-Hash SpMSpV	PM-BHash*	C1
	LB-PM-BHash*	C2
CPU-SPA SpMSpV	PB-MSPA*	C3
	LB-PB-MSPA*	C4
	SPA-Based Gustavson [4]	C5
GPU SpMSpV	Masked Sort-Based [19]	G1
GPU SpMV	Masked CSR Vector [28]	G2
	Masked Merge-Based [20]	G3

pre-existing approaches were also evaluated. The criteria for selecting these algorithms are as follows:

- *Sparse Format*: To avoid overheads from sparse matrix format conversions (e.g., CSR5 [29], yaSpMV [30]), all candidate algorithms must rely on either the Compressed Sparse Row/Column (CSR/ CSC).
- *Mask Fusion*: Traditional SpMSpV and SpMV algorithms typically disregard mask operations. Thus, the chosen algorithms must exhibit high mask fusion efficiency to fully leverage the sparsity of the mask. For this reason, existing methods have been optimized and rewritten to integrate mask fusion.
- *Broad Applicability*: The selected algorithms must ensure robust performance across varying sparsity scenarios.

Table I summarizes the candidate algorithms. For the CPU, two types of algorithms were implemented: CPU-Hash SpMSpV and CPU-SPA SpMSpV, using hash-based and SPA-based approaches, respectively. The SPA-based approaches include our proposed bucket-based method as well as the masked SPA-based Gustavson algorithm used in SuiteSparse [4]. Since the Gustavson algorithm inherently incorporates load balancing, we refer to it as LB-MSPA for brevity. The SpMV was excluded from CPU candidates, as its performance is significantly better suited to GPU execution. As illustrated in Fig.5, GPU-based SpMV consistently achieves lower runtimes compared to CPU-based implementations.

For the GPU, the selected candidates include SpMSpV algorithms based on sorting methods [19], which are critical for achieving optimal execution paths, especially in low sparsity scenarios. These SpMSpV methods complement CPU implementations when the matrix sparsity is lower. Additionally, two GPU SpMV algorithms were included:

- *Masked CSR Vector* [28]: A straightforward approach that excels when matrix elements are evenly distributed.
- *Masked Merge-Based* [20]: This method uses a 2D merge-path strategy to achieve effective load balancing, making it well-suited for matrices with uneven distributions of non-zero elements.

Fig.10 compares the performance of these four algorithm categories. As shown, the proposed CPU-Hash SpMSpV algorithm performs optimally in high sparsity scenarios, particularly when *masked_valid_nnz* is low (corresponding to low *x_density* and *m_density*). For larger computation tasks, GPU SpMV

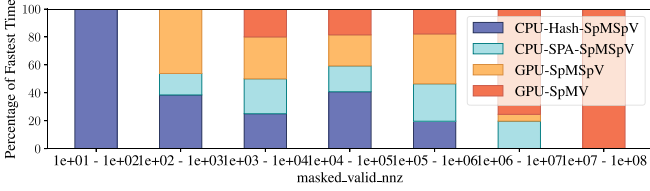
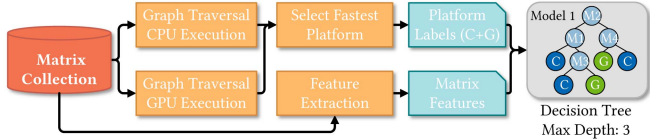
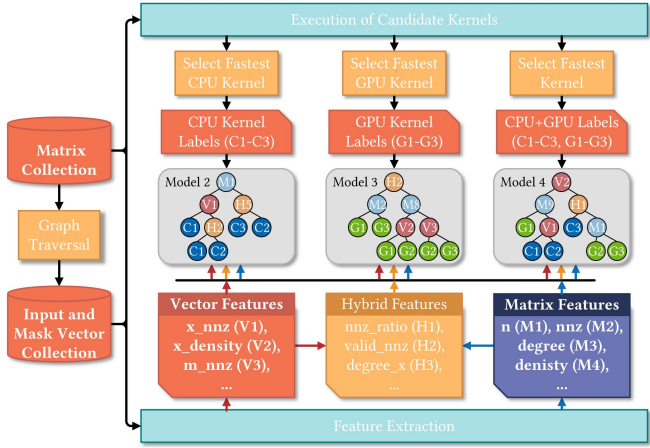


Fig. 10. Percentage of Optimality Achieved by Various Algorithms. (see Section V for experimental setup).



(a) Training Procedure of Model 1 (Platform Selector). ‘C’ and ‘G’ respectively denote the selected platforms for CPU and GPU.



(b) Training Procedure of Model 2, 3, and 4 (CPU Kernel Selector, GPU Kernel Selector, and Universal Kernel Selector). CPU kernels are denoted as ‘C1’-‘C3’, while GPU kernels as ‘G1’-‘G3’.

Fig. 11. Illustration of Key Training in CGA. Here, V, M, and H stand for vectors, matrices, and hybrid features, respectively.

outperforms all other methods. In intermediate sparsity ranges, where performance among algorithms is comparable, a more precise selection mechanism is necessary to identify the optimal algorithm.

C. Model Training

The model training process involves developing four classification models trained with labeled data using supervised learning [31], as illustrated in Fig. 11. For this task, we employ decision tree models [32], chosen for their simplicity and efficiency.

Model 1 follows a distinct training process, as shown in Fig. 11(a). Its training involves performing graph traversals on various datasets using both CPU and GPU implementations. By comparing the execution times of the two approaches, the faster execution path is assigned as the training label. This label, combined with matrix features, constitutes the training dataset

for Model 1. During training, kernel selection is performed by running all candidate kernels and selecting the one with the shortest execution time.

The training of Models 2, 3, and 4, as depicted in Fig. 11(b), involves conducting multiple graph traversals across a suite of sparse matrices. These traversals generate diverse input and mask vectors, from which features are extracted. The extracted features—derived from matrices, input vectors, and mask vectors—are then cross-combined to create a comprehensive feature set. Labels for these models are determined by running candidate kernels and selecting the one with the shortest execution time. This process results in a labeled dataset for kernel selection.

In the next subsection, we discuss the feature set and the specific details of the model training process.

1) *Features*: The features utilized by the classification models are categorized into three types: matrix features, vector features, and hybrid features, as outlined in Table II. Among the four decision models, a key step in the framework is the transition from GPU to CPU execution, which is governed by Model 4. This model determines whether such a transition is necessary by predicting whether the next kernel invocation should be executed on the CPU or GPU. To make accurate decisions, Model 4 must consider both the structural characteristics of the input graph (matrix features) and the dynamic state of the current iteration, including the number of active vertices (input vector) and visited vertices (mask vector). As a result, both matrix-level and vector-level information must be available during the training process to ensure the model captures the relevant patterns that influence the platform switching decision.

Matrix features include basic information, the non-zero distribution across rows, and across columns. These features are widely used in prior work because they capture key aspects of the problem’s scale, computational complexity, and memory-access characteristics. The distribution of non-zero elements in rows and columns helps determine whether load balancing should be applied. The Gini coefficient, typically used to measure income inequality, is applied here to assess the uniformity of non-zero element distribution [33].

Vector features include the number of non-zero elements and their densities in both the input vector and the mask. As analyzed in the motivation section, these metrics are used to assess the optimal timing for direction optimization and to evaluate the computational throughput of the SpMSPV.

Hybrid features, created through feature engineering, combine matrix and vector information. While the aforementioned features already encapsulate all the essential information for kernel selection, hybrid features introduce nonlinear combinations of existing features, which enhance classification accuracy and reduce decision tree depth, thus lowering runtime overhead.

Some of these hybrid features are derived from previous heuristic criteria, which manage the SpMSPV/SpMV switch. For example, Beamer et al. [17] and Gunrock [23] use the ratio of the number of neighbors of frontier vertices $|E_f|$ to the number of unvisited vertices’ neighbors $|E_u|$, approximately represented as

TABLE II
OVERVIEW OF FEATURES USED FOR MODEL CLASSIFICATION

Category	Features	Meaning	Model	Complexity	Abbr.
Matrix Basic	n	number of rows or columns	1,2,3,4	$\mathcal{O}(1)$	M1
	nnz	number of matrix nonzeros	1,2,3,4	$\mathcal{O}(1)$	M2
	degree (d)	average number of nonzeros per vertice	1,2,3,4	$\mathcal{O}(1)$	M3
	density	ratio of nonzeros of the matrix	1,2,3,4	$\mathcal{O}(1)$	M4
Row Distribution	row_max	maximum number of nonzeros per row	1,3,4	$\mathcal{O}(n)$	M5
	row_min	minimum number of nonzeros per row	1,3,4	$\mathcal{O}(n)$	M6
	row_relative_range	$(\text{row_max} - \text{row_min})/n$	1,3,4	$\mathcal{O}(n)$	M7
	row_Gini	Gini coefficient of row nonzeros	1,3,4	$\mathcal{O}(n \log(n))$	M8
Column Distribution	col_max	maximum number of nonzeros per column	1,2,4	$\mathcal{O}(n)$	M9
	col_min	minimum number of nonzeros per column	1,2,4	$\mathcal{O}(n)$	M10
	col_relative_range	$(\text{col_max} - \text{col_min})/n$	1,2,4	$\mathcal{O}(n)$	M11
	col_Gini	Gini coefficient of column nonzeros	1,2,4	$\mathcal{O}(n \log(n))$	M12
Vector Basic	x_nnz	number of nonzeros of the input vector	2,3,4	$\mathcal{O}(1)$ or $\mathcal{O}(n)$	V1
	x_density	ratio of nonzeros of the input vector	2,3,4	$\mathcal{O}(1)$ or $\mathcal{O}(n)$	V2
	m_nnz	number of nonzeros of the mask vector	2,3,4	$\mathcal{O}(1)$ or $\mathcal{O}(n)$	V3
	m_density	ratio of nonzeros of the mask vector	2,3,4	$\mathcal{O}(1)$ or $\mathcal{O}(n)$	V4
Hybrid Feature	nnz_ratio	x_nnz / m_nnz	3,4	$\mathcal{O}(1)^*$	H1
	valid_nnz	$nnz \times x_density$	2,3,4	$\mathcal{O}(1)^*$	H2
	masked_valid_nnz	$nnz \times x_density \times m_density$	2,3,4	$\mathcal{O}(1)^*$	H3
	degree_x	$degree \times x_density$	2	$\mathcal{O}(1)^*$	H4
	degree_x_m	$degree \times x_density \times m_density$	2	$\mathcal{O}(1)^*$	H5

$|E_f| \approx d|V_f|$ and $|E_u| \approx d|V_u|$, where d is the average degree of the graph, and $|V_f|$ and $|V_u|$ represent the number of frontier and unvisited vertices, respectively. So we have:

$$\frac{|E_f|}{|E_u|} \approx \frac{d|V_f|}{d|V_u|} = \frac{|V_f|}{|V_u|} = \frac{x_nnz}{m_nnz} = nnz_ratio \quad (1)$$

Ligra [34] and GraphBLAST [12] use the ratio of the number of neighbors of frontier vertices to the total number of edges $|E|$ as a criterion. So we also have:

$$\frac{|E_f|}{|E|} \approx \frac{d \times x_nnz}{nnz} = \frac{x_nnz}{n} = x_density \quad (2)$$

Since these works use $|E_f|$, we have included it as well:

$$|E_f| \approx d \times x_nnz = nnz \times x_density = valid_nnz \quad (3)$$

Hybrid features should also be designed with consideration of the specific candidate algorithms; if new algorithms are added, corresponding hybrid features should be introduced. For Model 2, we defined two hybrid features to differentiate between our proposed bucket-based masked algorithms and LB-MSPA. The bucket-based methods require an additional bucketing step, with PM-BHash and PB-MSPA respectively incurring approximate bucketing complexities of $\mathcal{O}(\text{degree} \times x_nnz \times m_density)$ and $\mathcal{O}(\text{degree} \times x_nnz)$ (assuming uniform distribution of elements). This is because both methods need to place $\text{degree} \times x_nnz \times m_density$ and $\text{degree} \times x_nnz$ elements into buckets. In contrast, the traditional SPA method requires an initialization step with a complexity of $\mathcal{O}(n)$. Therefore, the ratio between these two complexities serves as a critical criterion for the decision. The hybrid features, degree_x and degree_x_m , are derived as follows:

$$(\text{degree} \times nnz_x) / n$$

$$= \text{degree} \times x_density = \text{degree_x} \quad (4)$$

$$(\text{degree} \times nnz_x \times m_density) / n$$

$$= \text{degree} \times x_density \times m_density$$

$$= \text{degree_x_m} \quad (5)$$

The computational complexity of feature extraction, shown in Table II, is a key factor. Matrix features are pre-computed before graph traversal and reused in multiple iterations. In contrast, vector features need to be computed during each iteration at runtime. In Table II, the complexity of vector feature extraction is provided for both sparse and dense formats. Hybrid features can be directly calculated once matrix and vector features are available with a complexity of $\mathcal{O}(1)^*$.

2) *Decision Tree Training Details:* We chose decision trees due to their sufficient accuracy (over 90%) and minimal inference overhead, which is particularly critical in our scenario where low-latency predictions are essential. Alternatives such as SVMs, Random Forests, and Neural Networks incur significantly higher runtime costs—often tens to hundreds of times slower [25], [35]. Given that our optimized CPU SpMSPV kernels can execute in as little as 10^{-2} ms, the prediction time must remain negligible to preserve overall performance gains.

For training, we used 400 graph datasets comprising 125 real-world graphs from the SuiteSparse Matrix Collection [36] and 275 synthetic graphs generated using RMAT [37] and RGG [38]. These datasets yielded 20,000 training samples through multiple graph traversals.

The decision tree model was implemented using the scikit-learn package [39] with the following parameters: criterion: “gini”, splitter: “best”, max_depth: range of [2,10]. To ensure that our kernel selector models effectively learns the performance gap of algorithms, we incorporated sample weighting.

The calculation formula is:

$$\text{Weight} = \exp \left(\frac{\beta}{k-1} \sum_{i=1, i \neq o}^k \left(\frac{T_i}{T_o} - 1 \right) \right), \quad \beta > 0 \quad (6)$$

where β is a scaling parameter, T_i is the execution time for each algorithm, and T_o is the execution time of the optimal algorithm. This formula assigns greater weight to cases where the optimal algorithm significantly outperforms others.

D. Inference Execution

After training, the four decision tree models are deployed within the one-copy execution flow to infer the optimal kernel during the runtime of graph algorithms. To enhance runtime efficiency, we implemented several optimizations. First, while reading the matrix file in Matrix Market Exchange (MTX) format from the disk and generating the CSR and CSC formatted sparse matrices, we concurrently extract the matrix features. These features are stored alongside the CSR and CSC formats, eliminating the need for redundant extraction. Second, the conversion between sparse and dense vector formats occurs only during the GPU Execution. We leverage the GPU to accelerate these conversions, and the data transfer overhead from GPU to CPU is minimized by using the sparse format. Third, Model 2,3 and Model 4 make predictions concurrently, avoiding the prediction delay associated with sequential execution.

V. EXPERIMENTAL ENVIRONMENT

The hardware platform for the experiments is a dual-socket server, equipped with two Intel Xeon Gold 6240 processors. The processor features 18 hardware cores, each with a base frequency of 2.60 GHz and a turbo frequency of 3.2 GHz. It supports six memory channels and utilizes DDR4-2933 memory, capable of achieving a theoretical maximum memory bandwidth of 140 GB/s. Additionally, the server is outfitted with 8 NVIDIA Tesla V100 GPUs. All devices are connected by NVlink, with a bidirectional bandwidth of 300 GB/s.

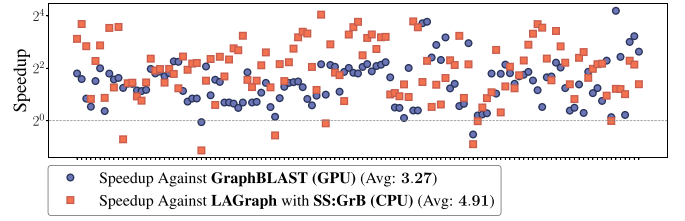
The software environment for the experiments was configured with Ubuntu 20.04, CUDA 12.2 and GCC 11.4.0, with the -O3 optimization option enabled. We used LAGraph version 1.1.1 and SuiteSparse:GraphBLAS version 8.2.1 to facilitate comparisons with state-of-the-art work.

VI. EXPERIMENT RESULTS

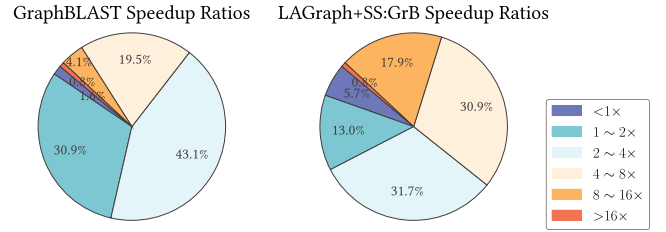
This section consists of five parts: (1) an overview of CGA's performance, an in-depth analysis of execution time gains from (2) masked SpMSPV and (3) adaptive selection strategies, and a study of how (4) prediction accuracy and (5) runtime overhead affect the framework.

A. Overall Performance of CGA

Baseline solutions include LAGraph [18] combined with SuiteSparse:GraphBLAS [4] (denoted as LAG+SS:GrB) for CPUs and GraphBLAST [12] for GPUs. SS:GrB is widely



(a) Speedup of CGA over GraphBLAST (GPU baseline) and LAGraph with SS:GrB (CPU baseline) across various graph datasets.



(b) Pie charts showing the distribution of CGA's speedup relative to each baseline.

Fig. 12. Overall performance of the proposed CGA framework on graph traversal tasks.

recognized as the fastest GraphBLAS implementation for multi-core processors, with LAGraph providing optimized graph algorithms built on SS:GrB. Conversely, GraphBLAST leads GPU-based GraphBLAS implementations, leveraging direction-optimized algorithms for improved performance.

The benchmark dataset comprises 128 graphs from the SuiteSparse Matrix Collection, focusing on real-world and synthetic graphs that reflect practical use cases. Fig. 12 illustrates the speedup achieved by our CGA framework compared to baseline approaches for graph traversal. On average, CGA delivers a $3.27\times$ speedup over GPU baseline and a $4.91\times$ speedup over CPU baseline. The speedup distribution (Fig. 12(b)) shows that CGA typically achieves a 1–4x speedup over GPU baseline and 2–16x speedup over CPU baseline.

However, in some cases, the speedup against CPU baseline is less than 1, particularly for small-scale mesh graphs. In such cases, SS:GrB efficiently utilizes fewer threads to execute, which CGA does not account for. Nonetheless, the slowdown on these graphs is acceptable, as their overall execution time is very short.

B. Performance of Masked SpMSPV

Figs. 13 and 14 compare the performance of the traditional SPA and our bucket-based masked SpMSPV algorithms on scale-free and mesh graphs, respectively, across a range of $x_density$ and $m_density$ values. These scenarios are chosen to highlight cases where CPU SpMSPV achieves optimal performance in BFS iterations. Our algorithms act as critical complements to traditional approaches, achieving higher performance in many cases.

On scale-free graphs with high Gini coefficients (indicating uneven non-zero element distributions), load-balanced variants

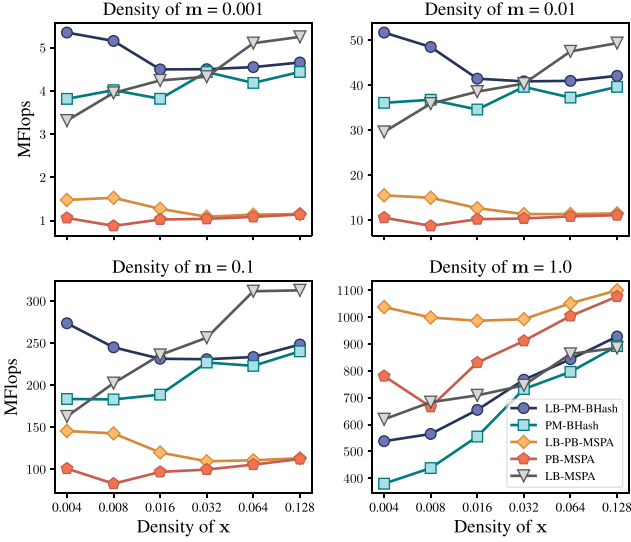


Fig. 13. Performance comparison between the proposed bucket-based SpM-SpV algorithms and traditional SPA methods, evaluated on the *kron_g500-logn21*, which is scale-free with a high average degree (86.8) and a high Gini coefficient (0.92).

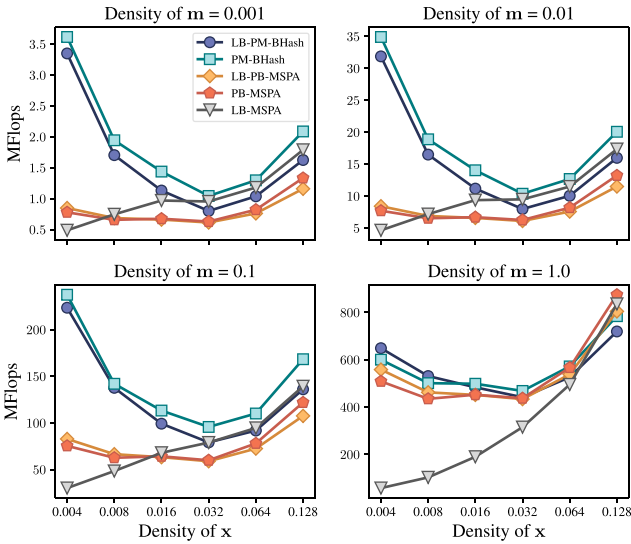


Fig. 14. Performance comparison on the *road_usa*, which is mesh graph with a low average degree (2.4) and a low Gini coefficient (0.21).

of our algorithms consistently outperform non-load-balanced ones. When comparing PM-BHash and PB-MSPA:

- At low $m_density$, PM-BHash performs better due to its reduced bucketing overhead (from pre-masking) and more compact hash table, which improves memory locality.
- As $m_density$ increases, the difference of bucketing overhead diminishes, and hash table size becomes comparable to SPA. At this point, SPA's direct indexing provides an advantage, enabling PB-MSPA to outperform PM-BHash. PB-MSPA's masking directly within SPA avoids additional accesses to the mask, further enhancing its performance.

When comparing PM-BHash and LB-MSPA:

TABLE III
OVERVIEW OF THE DATA SETS FOR EXPERIMENTS

	Data Sets	n ($\times 10^6$)	nnz ($\times 10^6$)	Gini	Degree
scale-free	soc-LiveJournal1	4.85	69.0	0.7232	14.2
	ljournal-2008	5.36	79.0	0.7394	14.7
	kron_g500-logn21	2.10	182	0.9167	86.8
	wikipedia-2007	3.57	45.0	0.7284	12.6
	hollywood-2009	1.14	114	0.7492	100
	flickr	0.821	9.84	0.8993	12.0
	soc-orkut	3.00	213	0.5586	71.0
	indochina-2004	7.41	194	0.7421	26.2
	wb-edu	9.85	57.2	0.7052	5.81
mesh	delaunay_n24	16.8	101	0.1220	6.01
	rgg_n_2_24_s0	16.8	133	0.1415	7.92
	belgium_osm	1.44	3.10	0.0901	2.15
	G3_circuit	1.59	4.62	0.0780	2.91
	roadNet-CA	1.97	5.53	0.1880	2.81
	hugetrace-00020	16.0	48.0	0.0002	3.00
	hugetric-00020	7.12	21.4	0.0003	3.01
	dieFilterV3real	1.10	45.2	0.2183	41.1
	road_usa	23.9	57.7	0.2107	2.41

- PM-BHash also outperforms LB-MSPA at low $m_density$ by avoiding the $\mathcal{O}(n)$ SPA initialization cost. However, its performance advantage diminishes at high $x_density$ or $m_density$, where bucketing overhead increases.

On mesh graphs, where average degrees are lower, the bucketing overhead is minimal, allowing our algorithms to achieve a significant performance advantage over the traditional SPA method. At low $x_density$, PM-BHash benefits from reduced data in each bucket and smaller hash table size, minimizing cache pressure and decreasing cache misses. Compared to LB-MSPA, PM-BHash can achieve up to a $10\times$ speedup. When $m_density=1$, both algorithms exhibit similar data distributions within each bucket, leading to comparable performance trends.

The results highlight the difficulty of selecting the optimal algorithm across diverse graphs and input conditions. In the next section, we demonstrate how CGA's adaptive selection effectively harnesses the performance potential of various algorithms in BFS.

C. Detailed Performance of CGA

We evaluated CGA on 18 representative graphs (9 scale-free and 9 mesh graphs) with diverse sizes and sparsity levels from the SuiteSparse Matrix Collection, as summarized in Table III. These graphs also serve as benchmark datasets established in works by Li et al. [25] Across these benchmarks, CGA achieved an average acceleration of $5.2\times$ over LAG+SS:GrB and $3.5\times$ over GraphBLAST.

Fig. 15 presents an ablation study illustrating the performance gains from improved kernel execution. Note that the measurements exclude other runtime overheads, resulting in slightly higher average speedups compared to those reported in Section VI-A. In each subfigure, the left panel reports the average improvement over 18 graphs, while the right panel shows per-graph results. Within each group, the leftmost bar denotes the

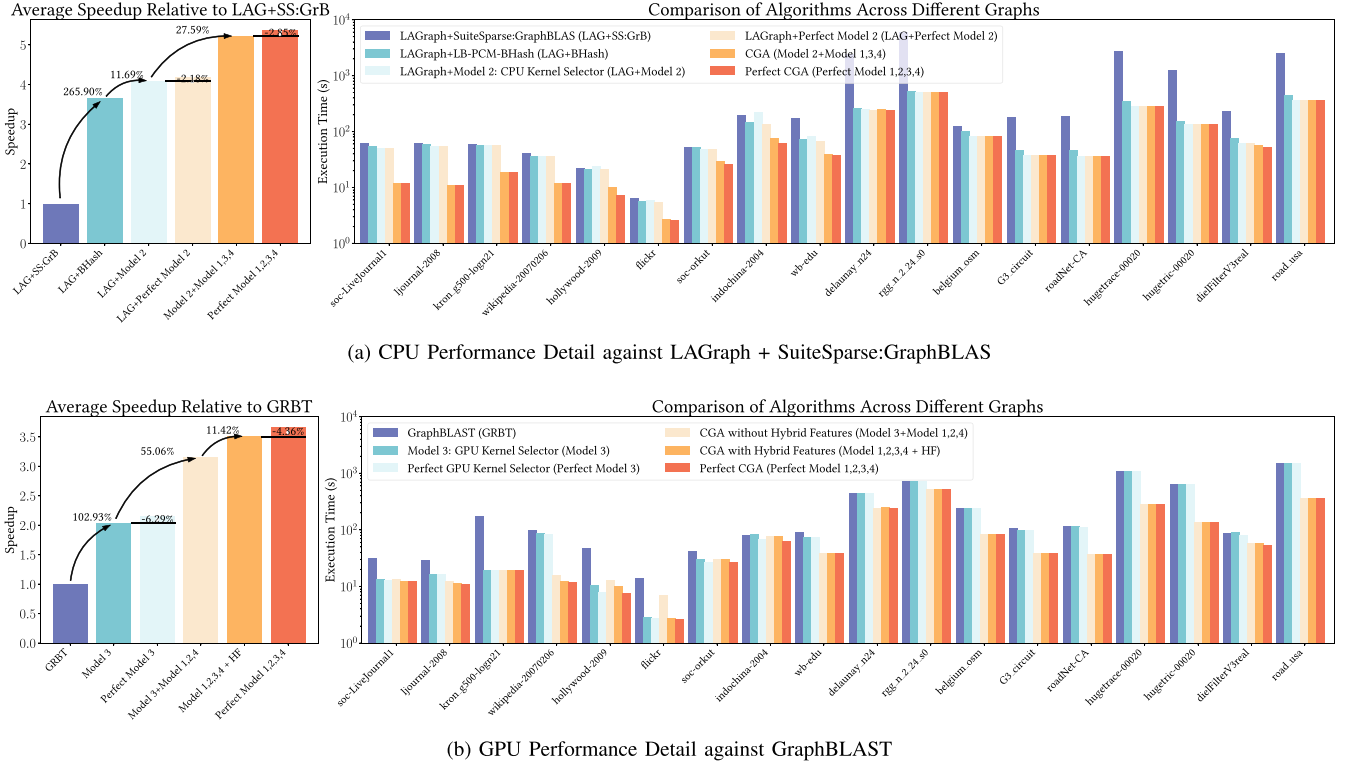


Fig. 15. Performance Benchmark: This Work against GraphBLAST and LAGraph + SuiteSparse:GraphBLAS.

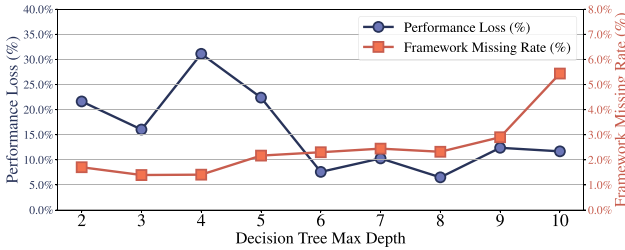


Fig. 16. Impact of Decision Tree Depth on Model Accuracy. **Framework Missing Rate** denotes the rate of non-optimal kernels. **Performance Loss** represents the performance degradation relative to the perfect framework.

baseline (i.e., LAG+SS:GrB and GRBT). The “Perfect Model” bars serve as an upper bound, assuming ideal predictions that always select the optimal kernel. While not achievable in practice, they help quantify the maximum potential gain. The small gap between our model and the perfect case suggests that the prediction accuracy is sufficiently high to ensure near-optimal performance.

As shown in Fig. 15(a), we first replaced the SS:GrB SpMSpV kernels in the CPU baseline (LAG+SS:GrB) with the LB-PM-BHash algorithm, resulting in a substantial performance gain of 265.9%, especially on mesh-like graphs where this kernel is frequently invoked. Next, we replaced the expert-rule-based direction selection strategy in LAGraph with the Model 2 CPU kernel selector, achieving an additional 11.69% improvement by leveraging learned decision-making. The performance closely approaches that of the “Perfect Model 2” indicating the model’s high prediction accuracy. Further gains come from

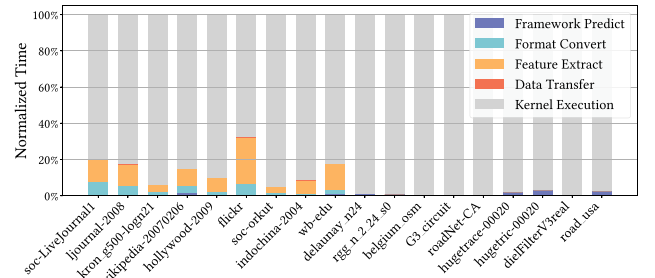


Fig. 17. Breakdown of CGA Runtime Overheads.

enabling the full CGA framework (Model 2 + Models 1, 3, 4), which incorporates GPU kernels through the one-copy execution flow, yielding an additional 27.59% improvement. The overall performance is only 2.85% lower than that of the perfect framework.

As shown in Fig. 15(b), the second bar illustrates that using Model 3 (the GPU kernel selector) alone improves performance by 102.93% through more accurate selection of GPU kernels. The performance gap compared to the “Perfect Model 3” is only 6.29%, highlighting the high accuracy of our model. These improvements are most notable on scale-free graphs, where correctly identifying the optimal switch point between SpMSpV and SpMV is critical. In this regard, Model 3 outperforms GraphBLAST. On mesh-like graphs, both GraphBLAST and Model 3 can reliably choose the appropriate GPU SpMSpV kernel.

The following bars show the impact of introducing CPU kernels through the One-Copy Execution Flow, enabled by

TABLE IV
DETAILED COMPARISON OF DIFFERENT RELATED WORKS ON THE CPU AND GPU

Capabilities	This Work	Framework					
		Beamer'12[17]	Gunrock[23]	LAGraph[18]	GraphBLAST[22]	SMAT[57]	Lin'21[25]
Adaptiveness	✓					✓	✓
Input Sparsity	✓	✓	✓	✓	✓		✓
Mask Sparsity	✓	✓	✓	✓	✓		
GraphBLAS-Ready	✓			✓	✓		
Backend	Both	CPU	GPU	CPU	GPU	CPU	GPU

combining Model 3 with Models 1, 2, and 4—collectively forming our full CGA framework. The right panel reveals that CGA brings notable speedups on mesh graphs due to the effective utilization of CPU-based SpMSpV.

To evaluate the effectiveness of the hybrid features described in Table II, the fourth and fifth bars compare the CGA framework without (Model 3 + Models 1, 2, 4) and with (Models 1,2,3,4 + HF) hybrid feature integration. Incorporating hybrid features yields an additional 11.42% speedup. Overall, the full CGA framework performs within 4.36% of the theoretical upper bound defined by the perfect model (Perfect Models 1,2,3,4), demonstrating the strength of our learned strategies and feature design.

D. Prediction Accuracy

Fig. 16 illustrates how prediction accuracy affects CGA's performance. With decision tree depths below 8, the framework maintains low miss rates. However, at depths below 6, a noticeable performance decline occurs, indicating that increased depth enhances the sample weighting's effectiveness (Formula 6). Beyond a depth of 8, overfitting leads to higher miss rates and performance losses. To balance depth and prediction overhead, a depth of 6 was consistently applied in this study.

E. Runtime Overhead

The runtime overhead of the framework impacts the acceleration. These overheads include: (1) predictions from the decision tree models, (2) vector format conversions, (3) feature extraction from vectors, and (4) potential single data transfers. Fig. 17 shows that runtime costs are generally low, with mesh graphs exhibiting significantly lower costs compared to scale-free graphs:

- Mesh graphs typically follow the CPU execution branch, where sparse vector formats dominate. As vector feature extraction complexity is $\mathcal{O}(1)$ (Table II), overhead is negligible. Thus, the only significant cost comes from decision tree predictions, which are minimal compared to kernel execution times.
- Scale-free graphs primarily follow the GPU execution branch, incurring all four overhead types. Feature extraction, which involves multiple $\mathcal{O}(n)$ calculations on vectors generated by SpMV, represents the largest cost. This overhead is particularly high for graphs like flickr, where frequent SpMV operations dominate runtime. Vector format conversion has a higher theoretical complexity ($\mathcal{O}(n + x_{nnz})$) but is required less frequently, resulting in lower total costs. In addition, concurrent predictions by Models 2, 3, and 4 further reduce prediction overhead to near-negligible levels.

VII. RELATED WORK

Research on SpMV has garnered significant attention, yielding notable performance enhancements. These include optimizations of the conventional COO/CSR formats [20], [21], [40], [41], [42], development of matrix formats tailored for specific matrix types or hardware architectures [29], [43], [44], [45], [46], [47], and selection of optimal computational methods based on matrix characteristics [25], [35], [48], [49], [50], [51], [52], [53].

In contrast to SpMV, SpMSpV has received relatively less attention. The research in this area has largely centered around direction-optimizing techniques in graph algorithms, such as BFS, to exploit input sparsity for performance gains [17]. Frameworks like Gunrock [23] have implemented these techniques on GPUs, while subsequent studies have refined GPU-based SpMSpV operations and improved push-pull strategies for better efficiency [19], [22]. On the CPU side, notable contributions include Ligra, GraphMat, and CombBLAS, which have optimized SpMSpV for shared memory environments [14], [15], [34], [55]. For multi-node distributed systems, GraphPad [56] offers an effective solution.

The introduction of mask sparsity in the GraphBLAS standard [57] has further advanced the field by defining graph algorithm operations using linear algebra. Implementations such as SuiteSparse [4], [13] have incorporated mask fusion for SpMV and SpMSpV on multicore CPUs, while GraphBLAST [22] extends similar capabilities to GPUs. LAGraph [18], built on SS:GrB, enhances graph analytics with direction-optimizing strategies, while GraphBLAST incorporates these optimizations natively. Other efforts include the development of parallel algorithms for masked SpGEMM, which compare different mask accumulators [58].

Machine learning-based adaptive approaches to sparse matrix operations date back to SMAT [54], which pioneered the selection of optimal sparse matrix formats for SpMV based on matrix characteristics, without leveraging input or output sparsity. Building on this foundation, subsequent studies explored adaptive selection mechanisms for SpMV in greater depth [35], [48], [49], [50], [51], [52]. Notably, Li et al. [25] expanded the scope of adaptive frameworks by incorporating SpMSpV into their algorithm candidate set, considering the effects of input sparsity on performance, though mask sparsity was not addressed in their work.

Despite these advances, existing research fails to address adaptiveness, input sparsity, and mask sparsity simultaneously. Moreover, current solutions are typically restricted to a single platform, either CPU or GPU, without consideration of CPU-GPU collaboration. In this context, our work seeks to

bridge these gaps by adhering to the GraphBLAS standard while implementing an adaptive framework that leverages both CPU and GPU resources. The distinctions between our approach and previous studies are summarized in Table IV, highlighting our contributions to advancing adaptive sparse computations.

VIII. CONCLUSION

In this paper, we presented CGA, a machine learning-based adaptive framework designed to autonomously select the optimal masked SpMSpV or SpMV kernels across CPU and GPU, thereby accelerating graph traversal. To harness the full potential of modern heterogeneous platforms, we first developed a set of bucket-based SpMSpV algorithms that deliver substantial performance improvements in scenarios of high input sparsity. We also curated a comprehensive set of candidate algorithms by integrating appropriate SpMSpV and SpMV methods tailored for both CPU and GPU execution. Leveraging machine learning, we built decision models capable of highly accurate kernel selection, and we further optimized the framework with a one-copy execution flow to minimize data transfer overhead between CPU and GPU.

Our experimental results demonstrate that CGA consistently outperforms CPU-only and GPU-only state-of-the-art approaches across a diverse set of real-world graphs. These improvements are driven by the inclusion of a broader and more effective candidate algorithm set and by precise decision-making enabled by our adaptive models. This work represents a pioneering effort to fully exploit the computational capabilities of both CPU and GPU for graph processing, achieving significant performance gains in the process.

Currently, since BFS consists solely of traversal operations with strict sequential dependencies, when one processing unit is active, the other remains idle. In real-world applications, traversal is often accompanied by additional computations. Coordinating traversal with these computations to effectively utilize heterogeneous resources presents a valuable direction for future work.

REFERENCES

- [1] H. Kwak, C. Lee, H. Park, and S. Moon, "What is Twitter, a social network or a news media?," in *Proc. 19th Int. Conf. World Wide Web*, New York, NY, USA, 2010, pp. 591–600, doi: [10.1145/1772690.1772751](#).
- [2] Y. Low, J. E. Gonzalez, A. Kyrola, D. Bickson, C. E. Guestrin, and J. Hellerstein, "GraphLab: A new framework for parallel machine learning," in *Proc. 26th Conf. Uncertainty Artif. Intell.*, 2010, pp. 340–349.
- [3] G. Malewicz et al., "PREGEL: A system for large-scale graph processing," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, New York, NY, USA, 2010, pp. 135–146, doi: [10.1145/1807167.1807184](#).
- [4] T. A. Davis, "Algorithm 1037: SuiteSparse: GraphBLAS: Parallel graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 49, no. 3, pp. 1–30, Sep. 2023, doi: [10.1145/3577195](#).
- [5] F. Zhang, J. Zhai, X. Shen, O. Mutlu, and X. Du, "POCLib: A high-performance framework for enabling near orthogonal processing on compression," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 459–475, Feb. 2022.
- [6] U. Sridhar, M. P. Blanco, R. Mayuranath, D. G. Spampinato, T. M. Low, and S. McMillan, "Delta-stepping SSSP: From vertices and edges to GraphBLAS implementations," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2019, pp. 241–250.
- [7] S. Ramachandran, "Incremental pagerank acceleration using sparse matrix-sparse vector multiplication," M.S. thesis, Dept. Comput. Sci. Eng., Ohio State Univ., Columbus, OH, USA, 2016.
- [8] E. Solomonik, M. Besta, F. Vella, and T. Hoefer, "Scaling betweenness centrality using communication-efficient sparse matrix multiplication," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage, Anal.*, New York, NY, USA, 2017, pp. 1–14, doi: [10.1145/3126908.3126971](#).
- [9] Graph500 Committee, "Graph 500: Benchmarking supercomputers with massive data," 2024. Accessed: Aug. 11, 2024. [Online]. Available: <https://graph500.org/>
- [10] S. Beamer, K. Asanović, and D. Patterson, "The gap benchmark suite," 2017, *arXiv:abs/1508.03619*.
- [11] G. Szárnyas, "Graphs and matrices: A translation of 'graphok és matrixok' by dénes könig (1931)," Sep. 2020, *arXiv:2009.03780*.
- [12] C. Yang, A. Buluç, and J. D. Owens, "GraphBlast: A high-performance linear algebra-based graph framework on the GPU," *ACM Trans. Math. Softw.*, vol. 48, no. 1, pp. 1–51, Feb. 2022, doi: [10.1145/3466795](#).
- [13] T. A. Davis, "Algorithm 1000: SuiteSparse: GraphBLAS: Graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 45, no. 4, pp. 1–25, Dec. 2019, doi: [10.1145/3322125](#).
- [14] N. Sundaram et al., "GraphMat: High performance graph analytics made productive," *Proc. VLDB Endowment*, vol. 8, no. 11, pp. 1214–1225, Jul. 2015, doi: [10.14778/2809974.2809983](#).
- [15] A. Azad, O. Selvitopi, M. T. Hussain, J. R. Gilbert, and A. Buluç, "Combinatorial BLAS 2.0: Scaling combinatorial algorithms on distributed-memory systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 4, pp. 989–1001, Apr. 2022.
- [16] H. Ji, H. Song, S. Lu, Z. Jin, G. Tan, and W. Liu, "TileSpMSpV: A tiled algorithm for sparse matrix-sparse vector multiplication on GPUs," in *Proc. 51st Int. Conf. Parallel Process.*, New York, NY, USA, 2023, pp. 1–11, doi: [10.1145/3545008.3545028](#).
- [17] S. Beamer, K. Asanović, and D. Patterson, "Direction-optimizing breadth-first search," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Washington, DC, USA, 2012, pp. 1–10.
- [18] G. Szárnyas et al., "LAGraph: Linear algebra, network analysis libraries, and the study of graph algorithms," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, Piscataway, NJ, USA, 2021, pp. 243–252.
- [19] C. Yang, Y. Wang, and J. D. Owens, "Fast sparse matrix and sparse vector multiplication algorithm on the GPU," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshop*, Piscataway, NJ, USA, 2015, pp. 841–847.
- [20] D. Merrill and M. Garland, "Merge-based sparse matrix-vector multiplication (SpMV) using the CSR storage format," in *Proc. 21st ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2016, pp. 1–2, doi: [10.1145/2851141.2851190](#).
- [21] Y. Niu, Z. Lu, M. Dong, Z. Jin, W. Liu, and G. Tan, "TileSpMV: A tiled algorithm for sparse matrix-vector multiplication on GPUs," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, Piscataway, NJ, USA, 2021, pp. 68–78.
- [22] C. Yang, A. Buluç, and J. D. Owens, "Implementing push-pull efficiently in GraphBLAS," in *Proc. 47th Int. Conf. Parallel Process.*, New York, NY, USA, 2018, pp. 1–11, doi: [10.1145/3225058.3225122](#).
- [23] Y. Wang et al., "Gunrock: GPU graph analytics," *ACM Trans. Parallel Comput.*, vol. 4, no. 1, pp. 1–49, Aug. 2017, doi: [10.1145/3108140](#).
- [24] L. Xu, H. Jia, Y. Zhang, L. Wang, and X. Jiang, "HAM-SpMSpV: An optimized parallel algorithm for masked sparse matrix-sparse vector multiplications on multi-core CPUs," in *Proc. 33rd Int. Symp. High-Perform. Parallel, Distrib. Comput.*, New York, NY, USA, 2024, pp. 160–173, doi: [10.1145/3625549.3658680](#).
- [25] M. Li, Y. Ao, and C. Yang, "Adaptive SpMV/SpMSpV on GPUs for input vectors of varied sparsity," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1842–1853, Jul. 2021.
- [26] A. Azad and A. Buluç, "A work-efficient parallel sparse matrix-sparse vector multiplication algorithm," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, Piscataway, NJ, USA, 2017, pp. 688–697.
- [27] M. T. Goodrich and R. Tamassia, *Algorithm Design and Applications*. Hoboken, NJ, USA: Wiley, 2014.
- [28] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proc. Conf. High Perform. Comput. Netw., Storage, Anal.*, New York, NY, USA, 2009, doi: [10.1145/1654059.1654078](#).
- [29] W. Liu and B. Vinter, "CSRS: An efficient storage format for cross-platform sparse matrix-vector multiplication," in *Proc. 29th ACM Int. Conf. Supercomput.*, New York, NY, USA, 2015, pp. 339–350, doi: [10.1145/2751205.2751209](#).
- [30] S. Yan, C. Li, Y. Zhang, and H. Zhou, "YASpMV: Yet another SpMV framework on GPUs," in *Proc. 19th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2014, pp. 107–118, doi: [10.1145/2555243.2555255](#).

- [31] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. Cambridge, MA: MIT Press, 2018.
- [32] C. D. Sutton, "Classification and regression trees, bagging, and boosting," in *Handbook of Statistics*, vol. 24. Amsterdam, The Netherlands: Elsevier, 2005, pp. 303–329.
- [33] J. Kunegis and J. Preusse, "Fairness on the web: Alternatives to the power law," in *Proc. 4th Annu. ACM Web Sci. Conf.*, New York, NY, USA, 2012, pp. 175–184, doi: [10.1145/2380718.2380741](https://doi.org/10.1145/2380718.2380741).
- [34] J. Shun and G. E. Blelloch, "LIGRA: A lightweight graph processing framework for shared memory," in *Proc. 18th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2013, pp. 135–146, doi: [10.1145/2442516.2442530](https://doi.org/10.1145/2442516.2442530).
- [35] Y. Zhao, J. Li, C. Liao, and X. Shen, "Bridging the gap between deep learning and sparse matrix format selection," in *Proc. 23rd ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2018, pp. 94–108, doi: [10.1145/3178487.3178495](https://doi.org/10.1145/3178487.3178495).
- [36] T. A. Davis and Y. Hu, "The university of florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1–25, 2011.
- [37] D. Chakrabarti, Y. Zhan, and C. Faloutsos, "R-Mat: A recursive model for graph mining," in *Proc. 2004 SIAM Int. Conf. Data Mining*, San Francisco, CA, USA, 2004, pp. 442–446.
- [38] M. Penrose, *Random Geometric Graphs*. vol. 5. Oxford, U.K.: OUP Oxford, 2003.
- [39] F. Pedregosa et al., "Scikit-Learn: Machine learning in python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [40] M. Steinberger, R. Zayer, and H.-P. Seidel, "Globally homogeneous, locally adaptive sparse matrix-vector multiplication on the GPU," in *Proc. Int. Conf. Supercomputing*, New York, NY, USA, 2017, doi: [10.1145/3079079.3079086](https://doi.org/10.1145/3079079.3079086).
- [41] G. Chu et al., "Efficient algorithm design of optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Perform. Parallel, Distrib. Comput.*, New York, NY, USA, 2023, pp. 115–128, doi: [10.1145/3588195.3593002](https://doi.org/10.1145/3588195.3593002).
- [42] X. You, C. Liu, H. Yang, P. Wang, Z. Luan, and D. Qian, "Vectorizing SpMV by exploiting dynamic regular patterns," in *Proc. 51st Int. Conf. Parallel Process.*, New York, NY, USA, 2023, pp. 1–12, doi: [10.1145/3545008.3545042](https://doi.org/10.1145/3545008.3545042).
- [43] C. Li, T. Xia, W. Zhao, N. Zheng, and P. Ren, "SpV8: Pursuing optimal vectorization and regular computation pattern in SpMV," in *Proc. 58th ACM/IEEE Des. Automat. Conf.*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 661–666.
- [44] M. B. Rajashekar, X. Tian, and Z. Fang, "HiSpMV: Hybrid row distribution and vector buffering for imbalanced SpMV acceleration on fpgas," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, New York, NY, USA, 2024, pp. 154–164, doi: [10.1145/3626202.3637557](https://doi.org/10.1145/3626202.3637557).
- [45] H. D. Tran, M. Fernando, K. Saurabh, B. Ganapathysubramanian, R. M. Kirby, and H. Sundar, "A scalable adaptive-matrix SpMV for heterogeneous architectures," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2022, pp. 13–24.
- [46] B. Xie et al., "CVR: Efficient vectorization of SpMV on X86 processors," in *Proc. Int. Symp. Code Gener. Optim.*, New York, NY, USA, 2018, pp. 149–162, doi: [10.1145/3168818](https://doi.org/10.1145/3168818).
- [47] C. Gómez, F. Mantovani, E. Focht, and M. Casas, "Efficiently running SpMV on long vector architectures," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2021, pp. 292–303, doi: [10.1145/3437801.3441592](https://doi.org/10.1145/3437801.3441592).
- [48] A. Benatia, W. Ji, Y. Wang, and F. Shi, "Sparse matrix format selection with multiclass SVM for SpMV on GPU," in *Proc. 45th Int. Conf. Parallel Process.*, 2016, pp. 496–505.
- [49] H. Cui, S. Hirasawa, H. Takizawa, and H. Kobayashi, "A code selection mechanism using deep learning," in *Proc. IEEE 10th Int. Symp. Embedded Multicore/Many-Core Syst.-on-Chip*, 2016, pp. 385–392.
- [50] Y. Zhao, W. Zhou, X. Shen, and G. Yiu, "Overhead-conscious format selection for SpMV-based applications," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2018, pp. 950–959.
- [51] I. Nisa, C. Siegel, A. S. Rajam, A. Vishnu, and P. Sadayappan, "Effective machine learning based format selection and performance modeling for SpMV on GPUs," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2018, pp. 1056–1065.
- [52] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, "Wise: Predicting the performance of sparse matrix vector multiplication with machine learning," in *Proc. 28th ACM SIGPLAN Annu. Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2023, pp. 329–341, doi: [10.1145/3572848.3577506](https://doi.org/10.1145/3572848.3577506).
- [53] Z. Du, J. Li, Y. Wang, X. Li, G. Tan, and N. Sun, "AlphaSparse: Generating high performance SpMV codes directly from sparse matrices," in *Proc. SC22: Int. Conf. High Perform. Computing, Netw., Storage Anal.*, 2022, pp. 1–15.
- [54] J. Li, G. Tan, M. Chen, and N. Sun, "SMAT: An input adaptive auto-tuner for sparse matrix-vector multiplication," *SIGPLAN Not.*, vol. 48, no. 6, pp. 117–126, Jun. 2013, doi: [10.1145/2499370.2462181](https://doi.org/10.1145/2499370.2462181).
- [55] A. Buluç and J. R. Gilbert, "The combinatorial BLAS: Design, implementation, and applications," *Int. J. High Perform. Comput. Appl.*, vol. 25, no. 4, pp. 496–509, 2011.
- [56] M. J. Anderson, N. Sundaram, N. Satish, M. M. A. Patwary, T. L. Willke, and P. Dubey, "Graphpad: Optimized graph primitives for parallel and distributed platforms," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, Piscataway, NJ, USA, 2016, pp. 313–322.
- [57] A. Buluç, T. Mattson, S. McMillan, J. Moreira, and C. Yang, "Design of the GraphBLAS API for C," in *Proc. IEEE Int. parallel Distrib. Process. Symp. Workshops*, 2017, pp. 643–652.
- [58] S. Milaković, O. Selvitopi, I. Nisa, Z. Budimlić, and A. Buluc, "Parallel algorithms for masked sparse matrix-matrix products," in *Proc. 51st Int. Conf. Parallel Process.*, New York, NY, USA, 2023, pp. 1–11, doi: [10.1145/3545008.3545048](https://doi.org/10.1145/3545008.3545048).



Lei Xu is currently working toward the PhD degree with the High Performance Computer Research Center, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, and also with the University of Chinese Academy of Sciences, China. His research interests include high-performance computing, heterogeneous parallel performance optimization.



Haipeng Jia received the PhD degree from the Ocean University of China, Qingdao, China, in 2012. From 2010 to 2012, he was a visiting PhD degree student with the Institute of Software, Chinese Academy of Sciences. He is currently an assistant professor with the High Performance Computer Research Center, Institute of Computing Technology, Chinese Academy of Sciences. His research interests include heterogeneous computing and many-core parallel programming method.



Yunquan Zhang (Senior Member, IEEE) received the PhD degree in computer software and theory from the Institute of Software Chinese Academy of Sciences, Beijing, China, in 2000. He is currently a full professor of computer science with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing. His research focuses on the areas of high-performance parallel computing, including large scale parallel computation and programming models, high-performance parallel numerical algorithms, and performance modeling and evaluation for parallel programs.